

เอกสารประกอบการสอน

Docker และการทำงานเบื้องต้น

(แนวคิดคอนเทนเนอร์ การสร้างอิมเมจ และการจัดการหลายบริการด้วย Docker Compose)

สำหรับนักศึกษาระดับปริญญาตรี เทคโนโลยีสารสนเทศ / วิทยาการคอมพิวเตอร์

คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยราชภัฏร้อยเอ็ด

สารบัญ

ส่วนที่ 1: ข้อมูลรายวิชาและวัตถุประสงค์การเรียนรู้.....	4
1.1 วัตถุประสงค์การเรียนรู้ (Learning Objectives — ตาม Bloom's Revised Taxonomy).....	4
ส่วนที่ 2: บทนำ — ปัญหา "ในเครื่องผมรันได้นี้ครับ"	5
ส่วนที่ 3: แนวคิดพื้นฐานของคอนเทนเนอร์	5
3.1 คอนเทนเนอร์ต่างจากเครื่องเสมือนอย่างไร.....	5
3.2 คำศัพท์หลัก 4 คำที่ต้องแยกให้ออก.....	6
3.3 กลไกชั้นข้อมูล (Layers) — หัวใจที่ทำให้ Docker เร็ว	6
ส่วนที่ 4: การติดตั้งและประเด็นใบอนุญาต	7
4.1 Docker Engine กับ Docker Desktop ไม่ใช่สิ่งเดียวกัน.....	7
4.2 การติดตั้ง Docker Engine บน Ubuntu / WSL2 Ubuntu	8
ส่วนที่ 5: คำสั่งพื้นฐานที่ใช้บ่อย	8
5.1 การทำงานกับอิมเมจ.....	9
5.2 การทำงานกับคอนเทนเนอร์	9
5.3 การทำความสะอาดพื้นที่	10
ส่วนที่ 6: การสร้างอิมเมจด้วย Dockerfile	10
6.1 คำสั่งหลักใน Dockerfile	10
6.2 ตัวอย่าง Dockerfile สำหรับเว็บแอปพลิเคชัน Python.....	11
6.3 ไฟล์ .dockerignore	11
6.4 แนวปฏิบัติที่ดีในการเขียน Dockerfile.....	12
6.5 การสร้างแบบหลายขั้น (Multi-stage Build).....	12
ส่วนที่ 7: การจัดเก็บข้อมูลและเครือข่าย	13
7.1 ข้อมูลในคอนเทนเนอร์หายเมื่อลบคอนเทนเนอร์.....	13
7.2 เครือข่ายและการที่คอนเทนเนอร์คุยกันเอง	13
ส่วนที่ 8: การจัดการหลายบริการด้วย Docker Compose	14
8.1 เหตุผลที่ต้องใช้ Compose.....	14

8.2 โครงสร้างไฟล์ compose.yaml	14
8.3 คำสั่ง Compose ที่ใช้บ่อย	15
8.4 การจัดการค่าความลับด้วยไฟล์ .env	15
ส่วนที่ 9: ปฏิบัติการรวม — ระบบเว็บแอปพลิเคชันกับฐานข้อมูล	16
ส่วนที่ 10: การบูรณาการ Generative AI	17
10.1 หลักคิด: AI ช่วยได้ดีกับไวยากรณ์ แต่ไม่รู้จักรบบของเรา	17
10.2 Workflow การใช้ AI ในปฏิบัติการนี้ (5 ขั้นตอน)	17
10.3 ตัวอย่าง Prompt ที่ใช้ได้จริง	18
ส่วนที่ 11: ความปลอดภัยและจริยธรรม	18
11.1 ประเด็นความปลอดภัยที่ต้องสอนตั้งแต่ต้น	18
11.2 ความซื่อสัตย์ทางวิชาการ	19
ส่วนที่ 12: กิจกรรมในชั้นเรียน (In-class Activities)	19
ส่วนที่ 13: แบบฝึกหัด/คำถามท้ายบท (พร้อมแนวคำตอบ)	20
13.1 คำถามทบทวน	20
13.2 แบบฝึกหัดปฏิบัติ (ส่งเป็นชิ้นงาน)	21
ส่วนที่ 14: เกณฑ์การประเมิน (Assessment Rubric)	21
ส่วนที่ 15: เอกสารอ้างอิงและแหล่งค้นคว้าเพิ่มเติม	22

ส่วนที่ 1: ข้อมูลรายวิชาและวัตถุประสงค์การเรียนรู้

รายการ	รายละเอียด
หัวข้อ	Docker และการใช้งานเบื้องต้น (Introduction to Docker and Containerization)
กลุ่มเป้าหมาย	นักศึกษาปริญญาตรี เทคโนโลยีสารสนเทศ / วิทยาการคอมพิวเตอร์ / วิศวกรรมซอฟต์แวร์
จำนวนคาบ	4-6 ชั่วโมง (บรรยาย 2 ชม. + ปฏิบัติการ 2-4 ชม.) หรือแบ่งเป็น 2 สัปดาห์
ความรู้พื้นฐาน	คำสั่งพื้นฐานบน Command Line, แนวคิดระบบปฏิบัติการเบื้องต้น, การพัฒนาเว็บแอปพลิเคชันและฐานข้อมูลเบื้องต้น
เครื่องมือที่ต้องเตรียม	เครื่องคอมพิวเตอร์ที่ติดตั้ง Docker Engine ได้ (Linux / Windows + WSL2 / macOS), บัญชี Docker Hub (สมัครฟรี), Text Editor เช่น VS Code

1.1 วัตถุประสงค์การเรียนรู้ (Learning Objectives — ตาม Bloom’s Revised Taxonomy)

เมื่อเรียนจบบทเรียนนี้ นักศึกษาจะสามารถ:

- (Remember)** ระบุความหมายของคอนเทนเนอร์ อิมเมจ และรีจิสทรี รวมถึงคำสั่ง Docker พื้นฐานที่ใช้อย่างน้อย
- (Understand)** อธิบายความแตกต่างระหว่างคอนเทนเนอร์กับเครื่องเสมือน (Virtual Machine) และอธิบายกลไกการทำงานแบบชั้น (layer) ของอิมเมจได้
- (Apply)** เขียน Dockerfile เพื่อบรรจุแอปพลิเคชันของตนเองเป็นอิมเมจ และรันเป็นคอนเทนเนอร์ที่เข้าถึงได้จริงได้
- (Analyze)** วิเคราะห์สาเหตุที่คอนเทนเนอร์ทำงานผิดพลาด จากบันทึกการทำงาน (logs) สถานะคอนเทนเนอร์ และการตั้งค่าเครือข่าย/พื้นที่จัดเก็บได้
- (Evaluate)** ประเมินคุณภาพและความปลอดภัยของ Dockerfile ตามแนวปฏิบัติที่ดี เช่น ขนาดอิมเมจ การใช้แคช และสิทธิ์ของผู้ใช้ในคอนเทนเนอร์ได้
- (Create)** ออกแบบและสร้างระบบหลายบริการ (เว็บแอปพลิเคชัน + ฐานข้อมูล) ด้วยไฟล์ Compose ที่รันได้ด้วยคำสั่งเดียวได้

ส่วนที่ 2: บทนำ — ปัญหา "ในเครื่องผมรันได้นี่ครับ"

สถานการณ์ที่นักศึกษาทุกคนเคยเจอ: โปรแกรมที่เขียนเสร็จและทดสอบผ่านบนเครื่องตนเอง กลับรันไม่ได้เมื่อส่งให้เพื่อนหรือติดตั้งบนเซิร์ฟเวอร์ สาเหตุคือ **ความแตกต่างของสภาพแวดล้อม (environment)** ได้แก่ เวอร์ชันของภาษาโปรแกรม ไบเบรารีที่ติดตั้งไว้ ตัวแปรระบบ ระบบปฏิบัติการ และการตั้งค่าต่าง ๆ ซึ่งไม่มีทางเหมือนกันทุกเครื่อง

แนวคิดของ Docker คือการแก้ปัญหานี้ที่ต้นเหตุ ด้วยการบรรจุ โปรแกรม + ไบเบรารี + สภาพแวดล้อมทั้งหมด ไว้ในหน่วยเดียวที่เรียกว่า **คอนเทนเนอร์ (Container)**

แล้วส่งมอบทั้งหน่วยนั้นไปทำงานที่ใดก็ได้ ผลลัพธ์คือสิ่งที่รันบนเครื่องนักศึกษา จะเป็นสิ่งเดียวกันกับที่รันบนเซิร์ฟเวอร์จริงทุกประการ

Docker เผยแพร่ครั้งแรกเมื่อวันที่ 20 มีนาคม ค.ศ. 2013 โดย Solomon Hykes เขียนด้วยภาษา Go และปัจจุบันกลายเป็นทักษะพื้นฐานที่อุตสาหกรรมซอฟต์แวร์คาดหวังจากบัณฑิตสายไอที เพราะเป็นรากฐานของงาน DevOps ระบบไมโครเซอร์วิส (Microservices) และการทำงานบนคลาวด์

เปรียบเทียบให้เห็นภาพ: การส่งของทางเรือเคยยุ่งยากเพราะสินค้าแต่ละชนิดมีรูปทรงต่างกัน จนกระทั่งมีการคิดค้น "ตู้คอนเทนเนอร์มาตรฐาน" ซึ่งทำให้เรือ รถบรรทุก และเครื่องบินไม่ต้องสนใจว่าข้างในเป็นอะไร ขอเพียงขนาดตู้เป็นมาตรฐานเดียวกัน — Docker ทำสิ่งเดียวกันกับซอฟต์แวร์

ส่วนที่ 3: แนวคิดพื้นฐานของคอนเทนเนอร์

3.1 คอนเทนเนอร์ต่างจากเครื่องเสมือนอย่างไร

ทั้งสองเทคโนโลยีสร้าง "สภาพแวดล้อมแยกส่วน" เหมือนกัน แต่แยกที่ระดับต่างกัน เครื่องเสมือน (VM) จำลองทั้งเครื่องรวมถึงระบบปฏิบัติการของตนเอง ส่วนคอนเทนเนอร์ใช้เคอร์เนลของระบบปฏิบัติการโฮสต์ร่วมกัน แล้วแยกเฉพาะส่วนที่จำเป็น ซึ่งเรียกเทคนิคนี้ว่า OS-level virtualization

ประเด็น	เครื่องเสมือน (VM)	คอนเทนเนอร์ (Container)
ระบบปฏิบัติการ	มี OS เต็มรูปแบบของตนเองในแต่ละเครื่อง	ใช้เคอร์เนลของโฮสต์ร่วมกัน
ขนาด	หลาย GB	ตั้งแต่ไม่กี่ MB ถึงหลายร้อย MB
เวลาเริ่มทำงาน	หลายสิบลวินาที ถึงนาที	ระดับวินาทีหรือน้อยกว่า

ระดับการแยกส่วน	แยกขาดในระดับฮาร์ดแวร์เสมือน จึงปลอดภัยกว่า	แยกที่ระดับกระบวนการ (process) เบากว่าแต่แยกขาดน้อยกว่า
เหมาะกับงาน	ต้องใช้ OS ต่างชนิด หรือต้องการความปลอดภัยระดับสูงสุด	ติดตั้งใช้งานแอปพลิเคชันจำนวนมาก ปรับขยายรวดเร็ว

ข้อควรระวังที่ต้องย้ำ: เพราะคอนเทนเนอร์ใช้เคอร์เนลของโฮสต์ร่วมกัน คอนเทนเนอร์ Linux จึงรันบนเคอร์เนล Linux เท่านั้น การรัน Docker บน Windows หรือ macOS จึงต้องอาศัยเครื่องเสมือน Linux เบื้องหลัง (เช่น WSL2 บน Windows) ซึ่งเป็นเหตุผลว่าทำไมประสิทธิภาพบนสองระบบนี้จึงต่างจาก Linux โดยตรง

3.2 คำศัพท์หลัก 4 คำที่ต้องแยกให้ออก

- **Image (อิมเมจ)** — แม่แบบแบบอ่านอย่างเดียว ที่บรรจุระบบไฟล์ โลบารี และคำสั่งเริ่มทำงานของแอปพลิเคชัน เปรียบได้กับ "คลาส" ในการเขียนโปรแกรมเชิงวัตถุ หรือ "แผ่นติดตั้งโปรแกรม"
- **Container (คอนเทนเนอร์)** — อิมเมจที่ถูกนำมารันจริงเป็นกระบวนการทำงาน เปรียบได้กับ "อ็อบเจกต์" ที่สร้างจากคลาส หนึ่งอิมเมจสร้างได้หลายคอนเทนเนอร์พร้อมกัน
- **Registry (รีจิสทรี)** — คลังเก็บอิมเมจสำหรับแบ่งปัน ที่นิยมที่สุดคือ Docker Hub เปรียบได้กับ GitHub แต่เก็บอิมเมจแทนซอร์สโค้ด
- **Dockerfile** — ไฟล์ข้อความที่เขียนคำสั่งบอกวิธีสร้างอิมเมจทีละชั้น เปรียบได้กับ "สูตรอาหาร" ที่ทำซ้ำได้ผลเหมือนเดิมทุกครั้ง

```

Dockerfile --( docker build )--> Image --( docker run )--> Container
                        ^   |
                        |   |
      ( docker pull )  |   |  ( docker push )
                        |   v
                        Registry (Docker Hub)

```

3.3 กลไกชั้นข้อมูล (Layers) — หัวใจที่ทำให้ Docker เร็ว

อิมเมจไม่ได้เป็นก้อนเดียว แต่ประกอบด้วย **ชั้น (layer)** หลายชั้นซ้อนกัน แต่ละคำสั่งใน Dockerfile สร้างขึ้นหนึ่งชั้น ชั้นเหล่านี้ถูกใช้ซ้ำและแคชไว้ ทำให้เกิดผลสำคัญสองข้อ:

- **ประหยัดพื้นที่** — อิมเมจหลายตัวที่ใช้ฐานเดียวกัน (เช่น ubuntu:24.04) จะใช้ชั้นฐานร่วมกัน ไม่เก็บซ้ำ

- **สร้างซ้ำได้เร็ว** — เมื่อแก้ไขแล้วสร้างใหม่ Docker จะใช้ชั้นเดิมจากแคชและสร้างใหม่เฉพาะชั้นที่เปลี่ยนแปลงและชั้นที่อยู่ถัดลงไปเท่านั้น

ผลที่ตามมาซึ่งต้องสอนตั้งแต่ต้น: ลำดับคำสั่งใน Dockerfile มีผลต่อความเร็วอย่างมาก
ให้วางสิ่งที่เปลี่ยนแปลงน้อย (เช่น การติดตั้งไลบรารี) ไว้ก่อน และสิ่งที่เปลี่ยนบ่อย (เช่น ซอร์สโค้ด) ไว้ทีหลัง

ส่วนที่ 4: การติดตั้งและประเด็นใบอนุญาต

4.1 Docker Engine กับ Docker Desktop ไม่ใช่สิ่งเดียวกัน

ส่วนประกอบ	รายละเอียด
Docker Engine	แกนหลักที่รันและจัดการคอนเทนเนอร์ ทำงานบน Linux เผยแพร่ภายใต้สัญญาอนุญาต Apache-2.0 (โอเพนซอร์ส) เวอร์ชันเสถียรล่าสุดอยู่ในสาย 29.x (29.7.1 เมื่อ 31 ก.ค. 2026)
Docker Desktop	โปรแกรมสำเร็จรูปพร้อมหน้าจอกกราฟิกสำหรับ Windows/macOS/Linux ซึ่งรวม Engine, CLI, Compose และเครื่องมืออื่นไว้ให้ ใช้สัญญาอนุญาตเชิงพาณิชย์ (Docker Subscription Service Agreement) ไม่ใช่โอเพนซอร์ส
Docker CLI	โปรแกรมบรรทัดคำสั่งที่ผู้ใช้พิมพ์คำสั่ง <code>'docker ...'</code> เพื่อสั่งงาน Engine
Docker Compose	ส่วนขยาย (plugin) สำหรับจัดการหลายคอนเทนเนอร์พร้อมกันด้วยไฟล์ตั้งค่าเดียว

ประเด็นใบอนุญาตที่อาจารย์และนักศึกษาต้องทราบ (สำคัญมาก)

Docker Desktop ให้ใช้ฟรีสำหรับการใช้งานส่วนบุคคล การศึกษา โครงการโอเพนซอร์สที่ไม่แสวงหากำไร และองค์กรขนาดเล็ก (พนักงานน้อยกว่า 250 คน และรายได้ต่อปีต่ำกว่า 10 ล้านดอลลาร์สหรัฐ)
องค์กรที่ใหญ่กว่านั้นต้องซื้อสิทธิ์การใช้งาน

อย่างไรก็ตาม ข้อตกลง SSA ฉบับปรับปรุงเมื่อ 10 ธันวาคม 2024 ได้จำกัดนิยาม
"การใช้เพื่อการศึกษาแบบฟรี" ให้ครอบคลุมเฉพาะ "นักศึกษาที่ลงทะเบียนเรียนในชั้นเรียน" เท่านั้น
มหาวิทยาลัยบางแห่งจึงตีความว่า อาจารย์ เจ้าหน้าที่ และนักวิจัย ต้องซื้อสิทธิ์การใช้งานหากจะใช้ Docker Desktop

ทางเลือกที่ปลอดภัยสำหรับการเรียนการสอน: ใช้ Docker Engine บน Linux โดยตรง (โอเพนซอร์ส ไม่มีข้อจำกัดนี้) หรือใช้ทางเลือกอื่นเช่น Podman หรือ Rancher Desktop — และควรตรวจสอบเงื่อนไขปัจจุบันจาก docker.com/pricing ก่อนเปิดภาคเรียนทุกครั้ง เพราะเงื่อนไขเปลี่ยนแปลงได้

4.2 การติดตั้ง Docker Engine บน Ubuntu / WSL2 Ubuntu

```
# ติดตั้งจากคลังทางการของ Docker (แนะนำ เพราะได้เวอร์ชันใหม่กว่าคลังของ Ubuntu)
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg

# เพิ่มกุญแจ GPG ของ Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# เพิ่มคลังซอฟต์แวร์ แล้วติดตั้ง Engine พร้อมส่วนขยายที่จำเป็น
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin

# ตรวจสอบผลการติดตั้ง
docker --version
docker compose version
sudo docker run hello-world
```

ขั้นตอนสำคัญหลังติดตั้ง: เพิ่มผู้ใช้เข้ากลุ่ม docker เพื่อไม่ต้องพิมพ์ sudo ทุกครั้ง

```
sudo usermod -aG docker $USER
# ต้องออกจากระบบแล้วเข้าใหม่ (หรือปิด-เปิด WSL) จึงจะมีผล
```

ประเด็นความปลอดภัยที่ต้องอธิบายให้นักศึกษาเข้าใจ: การอยู่ในกลุ่ม docker เทียบเท่ากับการมีสิทธิ์ระดับผู้ดูแลระบบ (root) บนเครื่องนั้น เพราะผู้ใช้สามารถรันคอนเทนเนอร์ที่ผูก (mount) ระบบไฟล์ทั้งเครื่องเข้ามาได้ จึงไม่ควรเพิ่มผู้ทั่วไปเข้ากลุ่มนี้บนเซิร์ฟเวอร์ที่ใช้งานจริงโดยไม่พิจารณา

ส่วนที่ 5: คำสั่งพื้นฐานที่ใช้บ่อย

5.1 การทำงานกับอิมเมจ

```
# ดาวน์โหลดอิมเมจจากรีจิสทรี (ถ้าไม่ระบุแท็ก จะได้ :latest)
docker pull nginx:1.27-alpine

# แสดงรายการอิมเมจในเครื่อง
docker images

# ค้นหาอิมเมจบน Docker Hub
docker search postgres

# ลบอิมเมจที่ไม่ใช้แล้ว
docker rmi nginx:1.27-alpine
```

อย่าใช้แท็ก latest ในงานจริง: แท็ก latest ไม่ได้แปลว่า "เวอร์ชันล่าสุด" เสมอไป แต่เป็นเพียงชื่อแท็กเริ่มต้น และอาจชี้ไปยังเวอร์ชันที่เปลี่ยนไปเมื่อใดก็ได้ ทำให้ระบบที่เคยรันได้พังโดยไม่มีการแก้ไข — ควรระบุเวอร์ชันชัดเจนเสมอ เช่น postgres:16-alpine

5.2 การทำงานกับคอนเทนเนอร์

```
# รันคอนเทนเนอร์แบบทำงานเบื้องหลัง พร้อมตั้งชื่อและแมปพอร์ต
docker run -d --name web -p 8080:80 nginx:1.27-alpine
#   -d          ทำงานเบื้องหลัง (detached)
#   --name      ตั้งชื่อให้ง่าย
#   -p 8080:80  พอร์ต 8080 ของเครื่องเรา -> พอร์ต 80 ในคอนเทนเนอร์

# ดูคอนเทนเนอร์ที่กำลังทำงาน (เพิ่ม -a เพื่อดูที่หยุดแล้วด้วย)
docker ps
docker ps -a

# ดูบันทึกการทำงาน (-f คือติดตามแบบต่อเนื่อง)
docker logs -f web

# เข้าไปใช้เชลล์ภายในคอนเทนเนอร์เพื่อตรวจสอบปัญหา
docker exec -it web sh

# หยุด / เริ่มใหม่ / ลบคอนเทนเนอร์
docker stop web
docker start web
docker rm -f web

# ดูการใช้ทรัพยากรแบบเรียลไทม์
docker stats
```

เคล็ดลับการวินิจฉัยปัญหา: เมื่อคอนเทนเนอร์ "รันแล้วดับทันที" ให้ใช้ `docker ps -a` ดูรหัสสถานะการออก (Exit Code) แล้วตามด้วย `docker logs <ชื่อ>` เพื่ออ่านข้อความผิดพลาดสุดท้ายก่อนดับ — สองคำสั่งนี้แก้ปัญหาได้เกินครึ่งของที่นักศึกษาพบ

5.3 การทำความสะอาดพื้นที่

```
# ลบคอนเทนเนอร์ที่หยุดแล้ว เครือข่ายที่ไม่ใช้ และอิมเมจที่ไม่มีใครอ้างถึง
docker system prune

# ดูว่าพื้นที่ถูกใช้ไปกับอะไรบ้าง
docker system df
```

คำเตือน: `docker system prune -a --volumes` จะลบข้อมูลในวอลุ่มที่ไม่ได้ใช้งานด้วย ซึ่งอาจทำให้ข้อมูลฐานข้อมูลหายถาวร ให้ระวังเป็นพิเศษ

ส่วนที่ 6: การสร้างอิมเมจด้วย Dockerfile

6.1 คำสั่งหลักใน Dockerfile

คำสั่ง	หน้าที่
FROM	กำหนดอิมเมจฐานที่จะต่อยอด ต้องเป็นคำสั่งแรกเสมอ
WORKDIR	กำหนดไดเรกทอรีทำงานภายในอิมเมจ (สร้างให้อัตโนมัติหากยังไม่มี)
COPY	คัดลอกไฟล์จากเครื่องเราเข้าไปในอิมเมจ
RUN	สั่งรันคำสั่ง "ขณะสร้างอิมเมจ" เช่น ติดตั้งไลบรารี ผลลัพธ์จะถูกบันทึกเป็นชั้นใหม่
ENV	กำหนดตัวแปรสภาพแวดล้อม
EXPOSE	ประกาศพอร์ตที่แอปพลิเคชันใช้ (เป็นเอกสารกำกับ ไม่ได้เปิดพอร์ตจริง ต้องใช้ -p ตอนรัน)
CMD	คำสั่งเริ่มทำงาน "ขณะรันคอนเทนเนอร์" มีได้หนึ่งคำสั่งต่อหนึ่งอิมเมจ และผู้ใช้เขียนทับได้ตอนรัน
ENTRYPOINT	คล้าย CMD แต่ผู้ใช้เขียนทับได้ยากกว่า เหมาะกับคอนเทนเนอร์ที่ทำหน้าที่เฉพาะอย่าง

จุดที่นักศึกษาสับสนบ่อยที่สุด: RUN ทำงาน "ตอนสร้างอิมเมจ" ส่วน CMD ทำงาน "ตอนรันคอนเทนเนอร์" — การเข้าใจผิดข้อนี้ทำให้เขียน Dockerfile ที่ติดตั้งไลบรารีใหม่ทุกครั้งที่เรา รัน หรือเขียนคำสั่งเริ่มเซิร์ฟเวอร์ไว้ใน RUN แล้วคอนเทนเนอร์ดับทันทีเมื่อรัน

6.2 ตัวอย่าง Dockerfile สำหรับเว็บแอปพลิเคชัน Python

```
# ใช้อิมเมจฐานที่ระบุเวอร์ชันชัดเจน และเลือกรุ่น slim เพื่อลดขนาด
FROM python:3.12-slim

# ตัวแปรสภาพแวดล้อมที่แนะนำสำหรับ Python ในคอนเทนเนอร์
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

# คัดลอกเฉพาะไฟล์รายการไลบรารีก่อน เพื่อให้ Docker ใช้แคชได้
# ตราบใดที่ requirements.txt ไม่เปลี่ยน ขั้นตอน pip install จะไม่ถูกรันซ้ำ
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ค่อยคัดลอกซอร์สโค้ดทีหลัง เพราะเป็นส่วนที่เปลี่ยนบ่อยที่สุด
COPY . .

# สร้างผู้ใช้ทั่วไปแล้วสลับไปใช้ เพื่อไม่ให้คอนเทนเนอร์รันด้วยสิทธิ์ root
RUN useradd --create-home appuser
USER appuser

EXPOSE 8000
CMD ["python", "app.py"]
```

คำสั่งสร้างและรันอิมเมจนี้:

```
# สร้างอิมเมจ (-t คือการตั้งชื่อและแท็ก, จุดท้ายคือบริบทการสร้าง)
docker build -t myapp:1.0 .

# รันเป็นคอนเทนเนอร์
docker run -d --name myapp -p 8000:8000 myapp:1.0
```

6.3 ไฟล์ .dockerignore

ไฟล์นี้ทำงานคล้าย .gitignore คือบอกว่าไฟล์ใดไม่ต้องส่งเข้าไปในกระบวนการสร้างอิมเมจ ช่วยให้สร้างเร็วขึ้น อิมเมจเล็กลง และ ป้องกันไฟล์ความลับหลุดเข้าไปในอิมเมจ

```
.git
```

```
.env
__pycache__/
*.pyc
node_modules/
venv/
*.log
README.md
```

6.4 แนวปฏิบัติที่ดีในการเขียน Dockerfile

- **ระบุเวอร์ชันอิมเมจฐานเสมอ** — ใช้ python:3.12-slim แทน python:latest เพื่อให้ผลลัพธ์ทำซ้ำได้
- **เรียงคำสั่งตามความถี่ของการเปลี่ยนแปลง** — สิ่งที่เปลี่ยนน้อยไว้บน สิ่งที่เปลี่ยนบ่อยไว้ล่าง เพื่อให้แคชทำงานได้เต็มที่
- **รวมคำสั่ง RUN ที่เกี่ยวข้องเข้าด้วยกัน** — แต่ละ RUN สร้างหนึ่งชั้น การใช้ && เชื่อมคำสั่งติดตั้งและล้างแคชในบรรทัดเดียวช่วยลดขนาดอิมเมจ
- **ไม่รันด้วยสิทธิ์ root** — สร้างผู้ใช้ทั่วไปแล้วใช้คำสั่ง USER เพราะโดยค่าเริ่มต้นคอนเทนเนอร์รันด้วยสิทธิ์ root ซึ่งเสี่ยงหากถูกเจาะ
- **ไม่ฝังความลับไว้ในอิมเมจ** — รหัสผ่านและกุญแจ API ต้องส่งผ่านตัวแปรสภาพแวดล้อมหรือกลไก secrets ตอนรัน ไม่ใช่เขียนใน Dockerfile เพราะทุกคนที่ได้อิมเมจไปจะอ่านได้
- **เลือกอิมเมจฐานให้เล็ก** — รุ่น alpine หรือ slim มีขนาดเล็กกว่ามาก ทำให้ดาวน์โหลดเร็วและมีช่องโหว่น้อยกว่า

6.5 การสร้างแบบหลายชั้น (Multi-stage Build)

เทคนิคนี้แยกขั้นตอน "สร้าง" ออกจากขั้นตอน "รัน" แล้วคัดลอกเฉพาะผลลัพธ์สุดท้ายไปยังอิมเมจที่เล็กที่สุด ทำให้อิมเมจสำหรับใช้งานจริงไม่มีเครื่องมือคอมไพล์ติดไปด้วย ซึ่งลดทั้งขนาดและความเสี่ยงด้านความปลอดภัย

```
# ----- ขั้นที่ 1: ขั้นสร้าง -----
FROM node:22-alpine AS builder
WORKDIR /src
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # ได้ผลลัพธ์ในโฟลเดอร์ /src/dist

# ----- ขั้นที่ 2: ขั้นใช้งานจริง -----
FROM nginx:1.27-alpine
# คัดลอกเฉพาะไฟล์ที่สร้างเสร็จแล้วจากขั้นที่ 1
COPY --from=builder /src/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

ส่วนที่ 7: การจัดเก็บข้อมูลและเครือข่าย

7.1 ข้อมูลในคอนเทนเนอร์หายเมื่อลบคอนเทนเนอร์

คอนเทนเนอร์ถูกออกแบบให้ **ไม่ถาวร (ephemeral)**

ข้อมูลที่เขียนลงในคอนเทนเนอร์จะหายไปเมื่อคอนเทนเนอร์ถูกลบ

ซึ่งเป็นบทเรียนที่นักศึกษา มักเรียนรู้ด้วยความเจ็บปวดจากการลบฐานข้อมูลของตนเอง

วิธีเก็บข้อมูลถาวรมีสองแบบหลัก:

รูปแบบ	ลักษณะ	เหมาะกับงาน
Volume	Docker บริหารจัดการพื้นที่ให้เอง อยู่นอกวงจรชีวิตคอนเทนเนอร์	ข้อมูลฐานข้อมูล ไฟล์ที่ผู้ใช้อัปโหลด — แนะนำสำหรับงานจริง
Bind mount	ผูกโฟลเดอร์บนเครื่องเราเข้ากับเส้นทางในคอนเทนเนอร์โดยตรง	การพัฒนา — แก้โค้ดบนเครื่องแล้วเห็นผลในคอนเทนเนอร์ทันที

```
# สร้างและใช้ volume สำหรับฐานข้อมูล
docker volume create pgdata
docker run -d --name db \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16-alpine

# ใช้ bind mount ระหว่างพัฒนา (ผูกโฟลเดอร์ปัจจุบันเข้ากับ /app)
docker run -d --name devapp -p 8000:8000 \
  -v "$(pwd)":/app \
  myapp:1.0

# ดูรายการและรายละเอียดวอลุ่ม
docker volume ls
docker volume inspect pgdata
```

7.2 เครือข่ายและการที่คอนเทนเนอร์คุยกันเอง

คอนเทนเนอร์ที่อยู่ในเครือข่ายเดียวกันสามารถเรียกหากันได้ด้วย **ชื่อคอนเทนเนอร์หรือชื่อบริการ** โดยตรง

เพราะ Docker มีบริการแปลงชื่อ (DNS) ภายในให้ —

นี่คือกลไกที่ทำให้เว็บแอปพลิเคชันเชื่อมต่อฐานข้อมูลได้โดยไม่ต้องรู้เลขที่อยู่ไอพี

```
# สร้างเครือข่ายของเราเอง แล้วให้สองคอนเทนเนอร์อยู่ในเครือข่ายเดียวกัน
docker network create appnet
```

```
docker run -d --name db --network appnet \
  -e POSTGRES_PASSWORD=secret postgres:16-alpine
```

```
docker run -d --name web --network appnet -p 8000:8000 myapp:1.0
```

```
# ภายในคอนเทนเนอร์ web สามารถเชื่อมต่อฐานข้อมูลด้วยชื่อโฮสต์ว่า "db"
# เช่น postgresql://postgres:secret@db:5432/postgres
```

ประเด็นอภิปราย: เหตุใดจึงไม่ควรใช้ localhost ในการเชื่อมต่อระหว่างคอนเทนเนอร์? — เพราะ localhost ภายในคอนเทนเนอร์หมายถึงตัวคอนเทนเนอร์นั่นเอง ไม่ใช่เครื่องโฮสต์หรือคอนเทนเนอร์อื่น การใช้ชื่อบริการจึงเป็นวิธีที่ถูกต้อง

ส่วนที่ 8: การจัดการหลายบริการด้วย Docker Compose

8.1 เหตุผลที่ต้องใช้ Compose

เมื่อระบบมีหลายส่วน เช่น เว็บแอปพลิเคชัน ฐานข้อมูล และแคช การพิมพ์คำสั่ง docker run หลายบรรทัดพร้อมพารามิเตอร์ยาวเหยียดทุกครั้งย่อมไม่สะดวกและเกิดข้อผิดพลาดได้ง่าย Docker Compose แก้ปัญหานี้ด้วยการให้เขียนทุกอย่างไว้ในไฟล์ YAML ไฟล์เดียว แล้วสั่งงานทั้งระบบด้วยคำสั่งเดียว

ข้อมูลอัปเดตสำคัญ (สิงหาคม 2026): Compose รุ่นที่ 1 (คำสั่ง docker-compose มีชิดกลาง ซึ่งเป็นโปรแกรมภาษา Python แยกต่างหาก) หมดการสนับสนุนแล้วตั้งแต่กลางปี 2023 ปัจจุบันต้องใช้ docker compose (เว้นวรรค ไม่มีชิดกลาง) ซึ่งเป็นส่วนขยายของ Docker CLI ที่เขียนด้วยภาษา Go นอกจากนี้ฟิลด์ version: ที่บรรทัดแรกของไฟล์ Compose **เลิกใช้แล้ว** หากยังใส่ไว้จะขึ้นคำเตือนให้ลบทิ้งได้เลย และชื่อไฟล์ที่แนะนำในปัจจุบันคือ compose.yaml (ยังรองรับชื่อเดิม docker-compose.yml อยู่) — สื่อการสอนเก่าจำนวนมากบนอินเทอร์เน็ตยังใช้รูปแบบเดิม ให้นักศึกษาระวังจุดนี้

8.2 โครงสร้างไฟล์ compose.yaml

```
services:
  web:
    build: . # สร้างอิมเมจจาก Dockerfile ในโฟลเดอร์นี้
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://appuser:${DB_PASSWORD}@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy # รอจนฐานข้อมูลพร้อมจริง ๆ
    restart: unless-stopped
```

```

db:
  image: postgres:16-alpine
  environment:
    POSTGRES_USER: appuser
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: appdb
  volumes:
    - pgdata:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s

volumes:
  pgdata:

```

จุดที่ต้องอธิบายเป็นพิเศษ — `depends_on` แบบธรรมดาจะรอเพียงให้คอนเทนเนอร์ "เริ่มทำงาน" เท่านั้น ไม่ได้รอให้ "พร้อมใช้งาน"

ทำให้เว็บแอปพลิเคชันมักพยายามเชื่อมต่อฐานข้อมูลก่อนที่ฐานข้อมูลจะเปิดรับ การใส่ `healthcheck` คู่กับ `condition: service_healthy` จึงเป็นวิธีแก้ปัญหานี้อย่างถูกต้อง

8.3 คำสั่ง Compose ที่ใช้บ่อย

```

# สร้างและเริ่มทุกบริการแบบทำงานเบื้องหลัง
docker compose up -d

# สร้างอิมเมจใหม่ก่อนเริ่ม (ใช้เมื่อแก้ Dockerfile หรือโค้ด)
docker compose up -d --build

# ดูสถานะและบันทึกการทำงาน
docker compose ps
docker compose logs -f web

# รันคำสั่งในบริการที่กำลังทำงาน
docker compose exec db psql -U appuser -d appdb

# หยุดและลบคอนเทนเนอร์ทั้งหมด (วอลุ่มยังอยู่)
docker compose down

# หยุดและลบพร้อมวอลุ่มด้วย <-- ข้อมูลฐานข้อมูลจะหายถาวร ใช้ด้วยความระมัดระวัง
docker compose down -v

```

8.4 การจัดการค่าความลับด้วยไฟล์ .env

สังเกตว่าในไฟล์ Compose ข้างต้นใช้ `{ DB_PASSWORD }` แทนการเขียนรหัสผ่านตรง ๆ ค่านี้จะถูกอ่านจากไฟล์ `.env` ในโฟลเดอร์เดียวกัน ซึ่ง **ต้องใส่ไว้ใน `.gitignore` เสมอ** และไม่อัปโหลดขึ้นระบบควบคุมเวอร์ชัน

```
# ไฟล์ .env (ห้าม commit ขึ้น Git เด็ดขาด)
DB_PASSWORD=ChangeMeToSomethingStrong
```

ส่วนที่ 9: ปฏิบัติการรวม — ระบบเว็บแอปพลิเคชันกับฐานข้อมูล

ปฏิบัติการนี้รวมทุกแนวคิดที่เรียนมาไว้ในโปรเจกต์เดียว ให้นักศึกษาสร้างโครงสร้างไฟล์ดังนี้

```
my-docker-lab/
├── compose.yaml
├── Dockerfile
├── .dockerignore
├── .env                # ไม่ commit
├── .gitignore
├── requirements.txt
└── app.py
```

ตัวอย่างแอปพลิเคชันอย่างง่ายที่นับจำนวนผู้เข้าชมและบันทึกลงฐานข้อมูล

เพื่อพิสูจน์ว่าข้อมูลอยู่รอดแม้รีสตาร์ทคอนเทนเนอร์:

```
# app.py
import os
from flask import Flask
import psycopg

app = Flask(__name__)
DB_URL = os.environ["DATABASE_URL"]

def init_db():
    with psycopg.connect(DB_URL) as conn:
        conn.execute("CREATE TABLE IF NOT EXISTS visits (
            \"id SERIAL PRIMARY KEY, seen_at TIMESTAMP DEFAULT
NOW() )")

@app.route("/")
def index():
    with psycopg.connect(DB_URL) as conn:
        conn.execute("INSERT INTO visits DEFAULT VALUES")
        total = conn.execute("SELECT COUNT(*) FROM visits").fetchone()[0]
    return f"<h1>ยินดีต้อนรับ</h1><p>มีผู้เข้าชมแล้ว {total} ครั้ง</p>"

if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=8000)
```


ขั้นตอนการทดสอบที่ต้องให้นักศึกษาทำครบ:

- 1. รัน docker compose up -d --build แล้วเปิดเบราว์เซอร์ที่ http://localhost:8000 รีเฟรชหลายครั้ง สังเกตตัวเลขเพิ่มขึ้น
- 2. รัน docker compose down (ไม่ใส่ -v) แล้ว up ใหม่ — ตัวเลขต้องไม่รีเซ็ต เพราะข้อมูลอยู่ในวอลุ่ม
- 3. รัน docker compose down -v แล้ว up ใหม่ — ตัวเลขต้องเริ่มจากศูนย์ เพื่อพิสูจน์ความแตกต่าง
- 4. ลองปิดบริการ dbชั่วคราวด้วย docker compose stop db แล้วรีเฟรชหน้าเว็บ สังเกตข้อความผิดพลาดและตรวจสอบด้วย docker compose logs web

ส่วนที่ 10: การบูรณาการ Generative AI

10.1 หลักคิด: AI ช่วยได้ดีกับไวยากรณ์ แต่ไม่รู้จักระบบของเรา

งาน Docker มีลักษณะที่ AI ช่วยได้ดีเป็นพิเศษ เพราะเป็นงานที่มีรูปแบบชัดเจนและมีตัวอย่างจำนวนมากในข้อมูลฝึก อย่างไรก็ตามมีข้อจำกัดสำคัญสองข้อ: (1) AI มักให้คำตอบที่ล้าสมัย เช่น แนะนำคำสั่ง docker-compose แบบมีขีดกลาง หรือใส่ฟิลด์ version: ที่เลิกใช้แล้ว เพราะข้อมูลฝึกมีเนื้อหาเก่าจำนวนมาก และ (2) AI ไม่ทราบบริบทระบบจริงของเรา เช่น เวอร์ชันไลบรารีที่ใช้ พอร์ตที่ว่างอยู่ หรือข้อกำหนดด้านความปลอดภัยขององค์กร

10.2 Workflow การใช้ AI ในปฏิบัติการนี้ (5 ขั้นตอน)

ขั้น	กิจกรรม	บทบาทของ AI
1	เขียน Dockerfile และ compose.yaml เอง	ไม่ใช่ AI — ต้องเข้าใจโครงสร้างระบบของตนเองก่อน
2	ตีความข้อความผิดพลาด	วางข้อความ error พร้อมบริบท (คำสั่งที่ใช้ ระบบปฏิบัติการ เวอร์ชัน Docker) ให้ AI อธิบายสาเหตุ
3	ตรวจทานเพื่อลดขนาดและเพิ่มความปลอดภัย	ขอให้ AI ชี้จุดที่ทำให้โหม่งใหญ่เกินไปเป็นประจำ แคลไม่ทำงาน หรือมีความเสี่ยงด้านสิทธิ์
4	ตรวจสอบความเป็นปัจจุบัน	ถาม AI ย้อนกลับว่าคำสั่งที่แนะนำยังใช้ได้กับ Docker รุ่นปัจจุบันหรือไม่ แล้วยืนยันกับเอกสารทางการอีกชั้น

5	ทดสอบจริงและบันทึก	ไม่ใช่ AI — ผลการรันจริงคือคำตัดสิน พร้อมบันทึก AI Log
---	--------------------	---

10.3 ตัวอย่าง Prompt ที่ใช้ได้จริง

Prompt 1 — วิเคราะห์ข้อความผิดพลาด:

"ผมรันคำสั่ง [ใส่คำสั่ง] บน Ubuntu 24.04 ด้วย Docker Engine เวอร์ชัน [ใส่เวอร์ชัน] แล้วได้ข้อความผิดพลาดนี้ [วาง error ทั้งหมด] นี่คือนโยบาย Dockerfile และ compose.yaml ของผม [วางไฟล์] ช่วยอธิบายว่าสาเหตุคืออะไร เกิดที่บรรทัดไหน และเสนอวิธีแก้ 2 แบบพร้อมข้อดีข้อเสีย โดยไม่ต้องเขียนไฟล์ใหม่ให้ทั้งหมด"

Prompt 2 — ตรวจสอบ Dockerfile:

"ทำหน้าที่เป็นวิศวกร DevOps ที่ตรวจสอบโค้ด ตรวจสอบ Dockerfile นี้ใน 4 ประเด็น: (1) ลำดับคำสั่งที่ทำให้แคชไม่ทำงาน (2) สิ่งที่ทำให้อิมเมจใหญ่เกินไป (3) ความเสี่ยงด้านความปลอดภัย เช่น การรันด้วย root หรือความลับที่ฝังอยู่ (4) การระบุเวอร์ชันที่ไม่ชัดเจน — ชี้เป็นรายบรรทัดพร้อมเหตุผล ห้ามเขียนไฟล์ใหม่ให้"

Prompt 3 — ตรวจสอบความเป็นปัจจุบัน (สำคัญมากสำหรับหัวข้อนี้):

"คำสั่งและไวยากรณ์ที่คุณเพิ่งแนะนำมา ยังเป็นแนวปฏิบัติปัจจุบันของ Docker หรือไม่ ช่วยระบุว่าส่วนใดอาจล้าสมัย เช่น การใช้ docker-compose แบบมีขีดกลาง หรือฟิลด์ version ในไฟล์ Compose พร้อมบอกว่าควรตรวจสอบจากหน้าเอกสารทางการไหนได้"

Prompt 4 — อธิบายแนวคิดด้วยการเปรียบเทียบ:

"อธิบายความแตกต่างระหว่าง volume กับ bind mount ให้นักศึกษาปี 3 ที่เพิ่งเริ่มเรียน Docker เข้าใจ โดยใช้การเปรียบเทียบกับสิ่งของในชีวิตประจำวัน พร้อมยกตัวอย่างสถานการณ์ที่เลือกใช้ผิดแล้วเกิดปัญหาอะไร"

ส่วนที่ 11: ความปลอดภัยและจริยธรรม

11.1 ประเด็นความปลอดภัยที่ต้องสอนตั้งแต่นั้น

- **ไม่ฝังความลับในอิมเมจ** — รหัสผ่านหรือกุญแจ API ที่เขียนใน Dockerfile จะติดอยู่ในชั้นข้อมูลของอิมเมจถาวร แม้จะลบในคำสั่งถัดไปก็ยังสามารถกู้คืนได้
- **ตรวจสอบที่มาของอิมเมจ** — อิมเมจจาก Docker Hub ที่ไม่ทราบที่มาอาจมีมัลแวร์หรือช่องโหว่ให้เลือกอิมเมจทางการ (Official Images) หรืออิมเมจจากผู้เผยแพร่ที่ยืนยันตัวตนแล้ว
- **อัปเดตอิมเมจฐานสม่ำเสมอ** — อิมเมจที่สร้างไว้นานย่อมมีช่องโหว่ที่ค้นพบภายหลัง ควรสร้างใหม่และสแกนหาช่องโหว่เป็นระยะ
- **จำกัดสิทธิ์ให้น้อยที่สุด** — ไม่รันด้วย root ไม่ใช่ตัวเลือก --privileged โดยไม่จำเป็น และไม่ผูกซ็อกเก็ตของ Docker เข้าไปในคอนเทนเนอร์ที่ไม่ไว้ใจ
- **ระงับการเปิดพอร์ต** — การแมปพอร์ตบนเครื่องที่มีโอเพิสสาธารณะเท่ากับเปิดบริการนั้นสู่อินเทอร์เน็ต ต้องตรวจสอบไฟร์วอลล์และการตั้งรหัสผ่านเสมอ

11.2 ความซื่อสัตย์ทางวิชาการ

- **เปิดเผยการใช้ AI (AI Log)** — ระบุเครื่องมือ prompt ที่ใช้ และส่วนที่ AI มีส่วนช่วยแนบท้ายรายงานทุกครั้ง
- **ต้องอธิบายทุกบรรทัดในไฟล์ที่ส่งได้** — ผู้สอนจะสุ่มถามความหมายของคำสั่งในการสอบปฏิบัติ การคัดลอกไฟล์มาโดยไม่เข้าใจถือว่าไม่ผ่าน
- **อ้างอิงแหล่งที่มาของอิมเมจและโค้ด** — อิมเมจและตัวอย่างโค้ดมีสัญญาอนุญาตกำกับ ต้องระบุที่มาและปฏิบัติตามเงื่อนไข

ส่วนที่ 12: กิจกรรมในชั้นเรียน (In-class Activities)

- **กิจกรรม 1 — คอนเทนเนอร์แรกใน 10 นาที (20 นาที):** ให้ทุกคนรัน nginx แล้วเปิดหน้าเว็บให้ได้ จากนั้นเปลี่ยนหมายเลขพอร์ตและตั้งชื่อคอนเทนเนอร์ให้ต่างกัน สังเกตว่าเกิดอะไรขึ้นเมื่อสองคอนเทนเนอร์ใช้พอร์ตเดียวกัน — เรียนรู้แนวความคิดการแมปพอร์ตจากข้อผิดพลาดจริง
- **กิจกรรม 2 — แข่งลดขนาดอิมเมจ (35 นาที):** แจก Dockerfile ที่เขียนไม่ดี (ใช้อิมเมจฐานเต็มรูปแบบ เรียงคำสั่งผิด รันด้วย root) ให้แข่งกันปรับปรุงให้อิมเมจเล็กที่สุดโดยที่โปรแกรมยังทำงานได้ วัดผลด้วยคำสั่ง docker images แล้วอภิปรายเทคนิคที่แต่ละกลุ่มใช้

- **กิจกรรม 3 — ลำบาก Compose (30 นาที):** ผู้สอนแจกโปรเจกต์ที่มีข้อผิดพลาดฝังไว้ 4 จุด (เช่น ใช้ localhost แทนชื่อบริการ, ลืมประกาศ volume, ระบุพอร์ตสลับด้าน, ใช้ไฟล์ version ที่เลิกใช้) ให้แข่งกันหาและแก้พร้อมบันทึกวิธีวินิจฉัย
- **กิจกรรม 4 — AI Review Lab (40 นาที):** ใช้ Prompt 2 และ 3 ในส่วนที่ 10 กับ Dockerfile ของตนเอง แล้วบันทึกผลตารางว่า AI ชี้จุดใดถูกต้อง จุดใดผิดหรือล้าสมัย และจุดใดที่ AI มองไม่เห็นแต่มนุษย์เห็น — ปิดท้ายด้วยการอภิปรายว่าเหตุใด AI จึงมักแนะนำไวยากรณ์เก่าในหัวข้อนี้
- **กิจกรรม 5 — ส่งต่อโปรเจกต์ (25 นาที):**
แต่ละกลุ่มส่งไฟล์เดอร์โปรเจกต์ให้กลุ่มอื่นทดลองรันบนเครื่องของตน
กลุ่มที่รับต้องรันให้สำเร็จด้วยคำสั่งเดียวโดยไม่ถามเจ้าของ — บททดสอบที่แท้จริงว่าโปรเจกต์ "ทำซ้ำได้" จริงหรือไม่

ส่วนที่ 13: แบบฝึกหัด/คำถามท้ายบท (พร้อมแนวคำตอบ)

13.1 คำถามทบทวน

1. อธิบายความแตกต่างระหว่างอิมเมจกับคอนเทนเนอร์ แนวตอบ:
อิมเมจคือแม่แบบแบบอ่านอย่างเดียวที่บรรจุระบบไฟล์และคำสั่งเริ่มทำงาน ส่วนคอนเทนเนอร์คืออิมเมจที่ถูกนำมารันเป็นกระบวนการจริง หนึ่งอิมเมจสร้างได้หลายคอนเทนเนอร์
2. เหตุใดคอนเทนเนอร์จึงเบาและเริ่มทำงานเร็วกว่าเครื่องเสมือน แนวตอบ:
เพราะใช้เคอร์เนลของระบบปฏิบัติการโฮสต์ร่วมกัน
ไม่ต้องบูตระบบปฏิบัติการของตนเองและไม่ต้องจำลองฮาร์ดแวร์
3. คำสั่ง RUN กับ CMD ใน Dockerfile ต่างกันอย่างไร แนวตอบ: RUN
ทำงานขณะสร้างอิมเมจและผลลัพธ์ถูกบันทึกเป็นชั้นข้อมูล ส่วน CMD คือคำสั่งเริ่มทำงานเมื่อรันคอนเทนเนอร์ และมีผลได้เพียงคำสั่งเดียวต่ออิมเมจ
4. เพราะเหตุใดจึงควรคัดลอกไฟล์ requirements.txt ก่อนคัดลอกซอร์สโค้ดทั้งหมด แนวตอบ:
เพื่อให้กลไกแคชของชั้นข้อมูลทำงาน — トラバิดที่รายการไลบรารีไม่เปลี่ยน
ขั้นตอนติดตั้งไลบรารีจะถูกใช้จากแคช ทำให้การสร้างอิมเมจซ้ำเร็วขึ้นมาก
5. Volume ต่างจาก Bind mount อย่างไร และควรใช้แบบใดกับฐานข้อมูล แนวตอบ: Volume ให้
Docker บริหารพื้นที่เอง เหมาะกับข้อมูลถาวรเช่นฐานข้อมูล ส่วน Bind mount
ผูกไฟล์เดอร์บนเครื่องเราโดยตรง เหมาะกับการพัฒนาที่ต้องแก้โค้ดแล้วเห็นผลทันที —
ฐานข้อมูลควรใช้ Volume

6. เหตุใดคอนเทนเนอร์จึงเรียกหา `localhost` ไม่ได้ แนวตอบ: `localhost` ภายในคอนเทนเนอร์หมายถึงตัวคอนเทนเนอร์นั่นเอง ต้องใช้ชื่อบริการหรือชื่อคอนเทนเนอร์ซึ่ง Docker แปลงเป็นเลขที่อยู่ไอพีให้ผ่านบริการ DNS ภายในเครือข่ายเดียวกัน
7. คำสั่ง `docker-compose` (มีขีดกลาง) กับ `docker compose` (เว้นวรรค) ต่างกันอย่างไรในปัจจุบัน แนวตอบ: แบบมีขีดกลางคือ Compose รุ่นที่ 1 ซึ่งหมดการสนับสนุนแล้ว ปัจจุบันต้องใช้แบบเว้นวรรคซึ่งเป็นส่วนขยายของ Docker CLI และควรลบไฟล์ `version`: ออกจากไฟล์ `Compose` ด้วย
8. (วิเคราะห์) นักศึกษารัน `docker compose up -d` แล้วบริการเว็บดับตันที่โดยฐานข้อมูลยังทำงานอยู่ จงเสนอลำดับขั้นการวินิจฉัย 4 ขั้น แนวตอบ: (1) `docker compose ps` ดูรหัสสถานะการออก (2) `docker compose logs web` อ่านข้อความสุดท้ายก่อนดับ (3) ตรวจสอบตัวแปรสภาพแวดล้อมและสายเชื่อมต่อฐานข้อมูลว่าใช้ชื่อบริการถูกต้องหรือไม่ (4) ตรวจสอบว่ามี `healthcheck` และ `condition: service_healthy` หรือไม่ เพราะอาจเชื่อมต่อก่อนฐานข้อมูลพร้อม

13.2 แบบฝึกหัดปฏิบัติ (ส่งเป็นชิ้นงาน)

- **แบบฝึกหัด A — บรรจุแอปพลิเคชันของตนเอง:** นำโปรเจกต์ที่เคยพัฒนาในรายวิชาอื่น (เว็บแอปพลิเคชันหรือ API) มาเขียน `Dockerfile`, `.dockerignore` และ `compose.yaml` ที่มีอย่างน้อย 2 บริการ (แอปพลิเคชัน + ฐานข้อมูล) พร้อม `volume` สำหรับข้อมูลถาวรและ `healthcheck` ส่งเป็นรีโพสิทอรี Git พร้อม README ที่อธิบายวิธีรันด้วยคำสั่งเดียว และแนบภาพผลการรัน
- **แบบฝึกหัด B — รายงานการปรับปรุงและการใช้ AI:** บันทึกตารางเปรียบเทียบขนาดอิมเมจก่อนและหลังการปรับปรุงพร้อมอธิบายว่าทำอะไรไป และบันทึกการแก้ปัญหาอย่างน้อย 3 ครั้งในรูปแบบ: อาการ / สมมติฐานของตนเอง / prompt ที่ใช้ / คำตอบของ AI / ผลตรวจสอบจริง (ถูกหรือผิด) / วิธีแก้สุดท้าย ปิดท้ายด้วยข้อสรุปครั้งหน้าว่า AI ให้คำตอบล้ำสมัยในเรื่องใดบ้าง

ส่วนที่ 14: เกณฑ์การประเมิน (Assessment Rubric)

เกณฑ์	ดีเยี่ยม (4)	ดี (3)	พอใช้ (2)
ความถูกต้องของ Dockerfile	สร้างสำเร็จ เรียงคำสั่งใช้แค่ได้เต็มที่ ระบุเวอร์ชันชัดเจน	สร้างสำเร็จ โครงสร้างถูกต้อง	สร้างได้แต่ลำดับคำสั่งไม่
การจัดการหลายบริการ (Compose)	รันด้วยคำสั่งเดียวได้ มี healthcheck และรอบริการพร้อมอย่างถูกต้อง	รันได้ครบทุกบริการ	รันได้แต่ต้องแก้ค่าด้วยมี

การจัดการข้อมูลถาวร	ใช้ volume ถูกต้อง พิสูจน์ได้ว่าข้อมูลอยู่รอดหลังรีสตาร์ท	ใช้ volume ได้ถูกต้อง	ใช้ผิดประเภทแต่ทำงาน
ความปลอดภัยและขนาดอิมเมจ	ไม่รันด้วย root ไม่มีความลับในอิมเมจ ขนาดเล็กเหมาะสม มี .dockerignore	ไม่มีความลับรั่ว ขนาดเหมาะสม	อิมเมจใหญ่เกินไปจำเป็น
การใช้ AI อย่างมีวิจารณญาณ	ตรวจพบและแก้คำแนะนำที่ล้าสมัยของ AI ได้ อ้างอิงเอกสารทางการ	ใช้ AI ช่วยและตรวจสอบผล	รับคำตอบ AI มาใช้เกือบ
จริยธรรมและเอกสาร (AI Log)	บันทึกครบ README ชัดเจน สะท้อนคิดลึก	บันทึกครบถ้วน	บันทึกไม่ครบ

การให้น้ำหนัก (ปรับได้ตามผู้สอน): ทักษะปฏิบัติหลัก (เกณฑ์ 1–3) 60% / ความปลอดภัยและการใช้ AI (เกณฑ์ 4–5) 25% / เอกสารและจริยธรรม (เกณฑ์ 6) 15%

ส่วนที่ 15: เอกสารอ้างอิงและแหล่งค้นคว้าเพิ่มเติม

- Docker Inc. Docker Documentation. <https://docs.docker.com/> — แหล่งอ้างอิงหลักที่ควรตรวจสอบก่อนเชื่อคำแนะนำจากแหล่งอื่น
- Docker Inc. Dockerfile Reference และ Best Practices for Writing Dockerfiles. <https://docs.docker.com/build/building/best-practices/>
- Docker Inc. Compose Specification. <https://docs.docker.com/reference/compose-file/>
- Docker Inc. Docker Subscription Service Agreement และหน้า Pricing. <https://www.docker.com/pricing/> — ตรวจสอบเงื่อนไขใบอนุญาตก่อนใช้ในหน่วยงาน
- Moby Project. <https://github.com/moby/moby> — ซอร์สโค้ดโอเพนซอร์สที่ Docker Engine สร้างขึ้นจาก
- Docker Hub. <https://hub.docker.com/> — คลังอิมเมจ ควรแนะนำนักศึกษาให้สังเกตเครื่องหมาย Official Image และ Verified Publisher
- OWASP. Docker Security Cheat Sheet — แนวปฏิบัติด้านความปลอดภัยสำหรับคอนเทนเนอร์

หมายเหตุสำหรับอาจารย์ผู้สอน: เอกสารนี้จัด heading hierarchy พร้อมสารบัญอัตโนมัติแล้ว (คลิกขวาที่สารบัญ > Update Field ใน Word) ข้อมูลเวอร์ชันและเงื่อนไขใบอนุญาตในเอกสารนี้อ้างอิง ณ

เดือนสิงหาคม 2026 ซึ่งเปลี่ยนแปลงบ่อย ควรตรวจสอบซ้ำก่อนเปิดภาคเรียนทุกครั้ง
และแนะนำให้เตรียมสภาพแวดล้อมสำรอง (เช่น เซิร์ฟเวอร์ Linux ส่วนกลางของคณะ
หรือบริการคลาวด์สำหรับการศึกษา) เพื่อกรณีเครื่องนักศึกษาติดตั้งไม่สำเร็จ ซึ่งมักเกิดกับเครื่อง Windows
ที่ยังไม่ได้เปิดใช้งาน WSL2 หรือการจำลองเสมือนใน BIOS